

Relatório 14 - Prática: Redes Neurais (II)

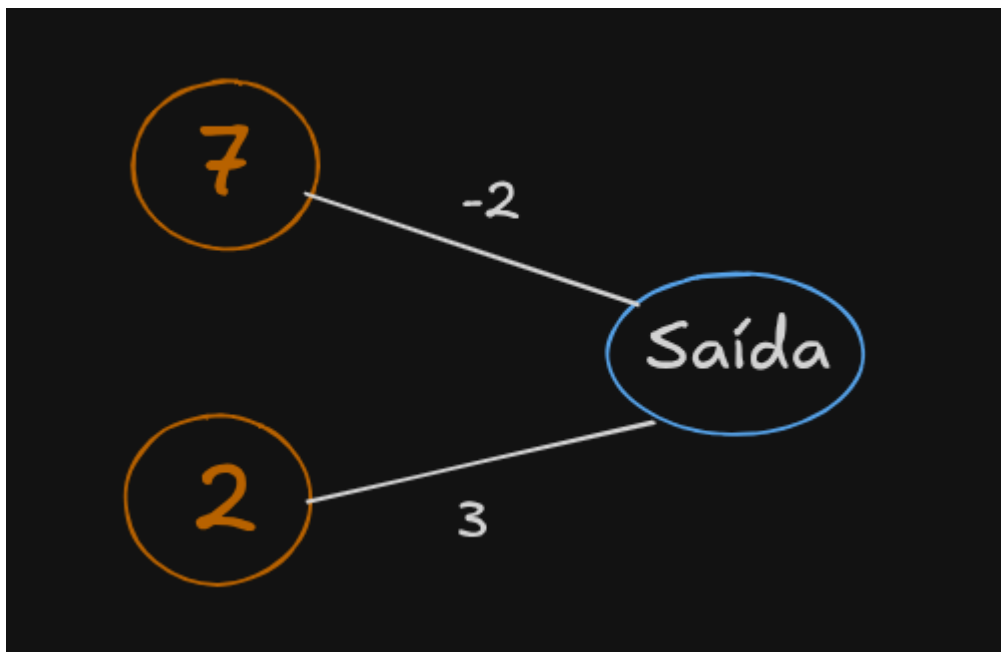
João Pedro Gomes

1. Introdução

O card 14 pede pra assistirmos vídeos onde é abordado o tema de redes neurais, os vídeos ensinam desde a teoria até a prática com as redes, eles ensinam as funções de ativação, como uma rede em si funciona, etc..., na parte prática ele ensina como criar uma rede em python e fazer previsões a partir de dados.

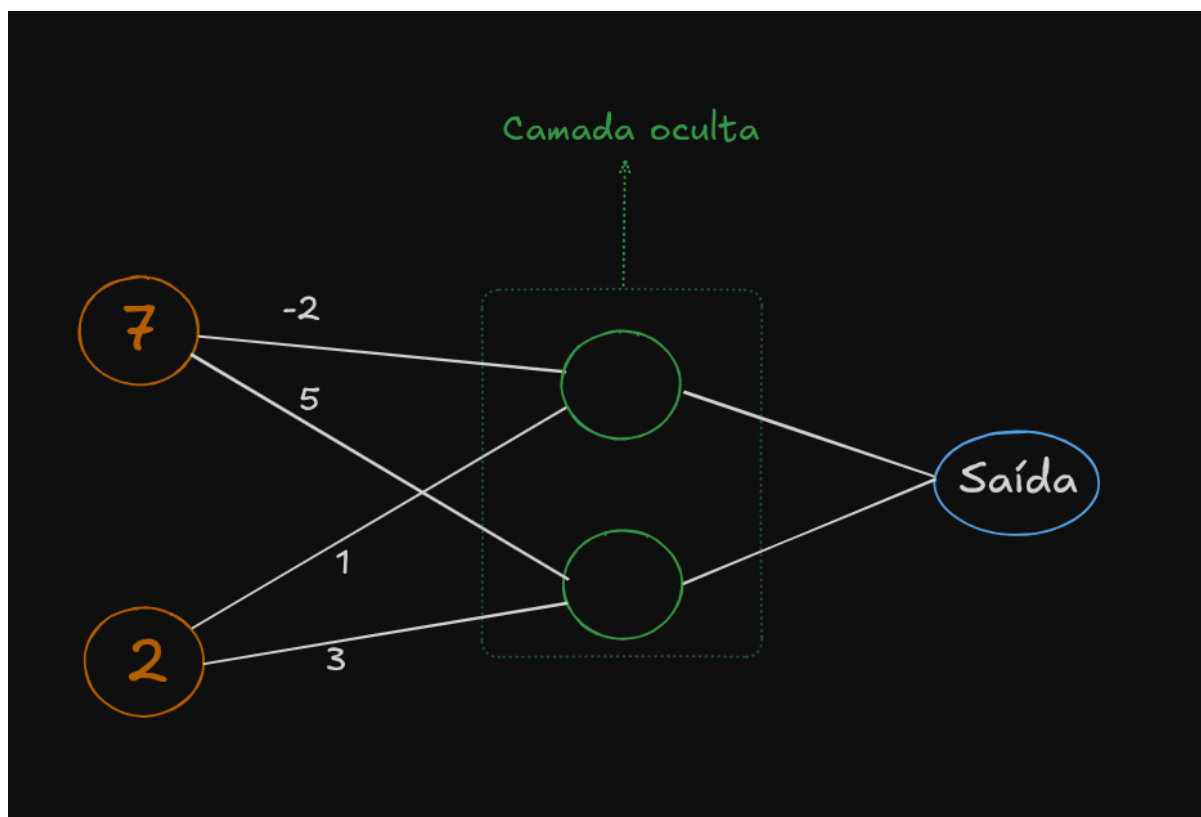
2. Desenvolvimento

Ele começa explicando o básico sobre um neurônio de uma rede neural, onde se tem as entradas, pesos e a saída:



Por exemplo, aqui temos as entradas 7 e 2, elas passam pelo peso na aresta e no final dependendo pra qual fim o neurônio serve ele tem sua devida saída.

E também tem a taxa de erro que ela vai se ajustando calculando o erro, que normalmente é o resultado previsto - o resultado real, isso vai se ajustando até ela errar o menos possível.



Aqui é uma rede em camadas densa, onde todos os neurônios de entrada são ligados ao da camada oculta, ela é mais usada pra coisas complexas, aqui é um exemplo fácil, a camada oculta processa os dados das arestas que são os pesos, faz a soma final e prevê na saída, supondo que seja uma step function que avalia o sinal, se for 0 ou menos ele sai 0, se for 1 ou maior que 0 ele sai 1, aqui ele sairia 0 e 1 pois no neurônio de cima daria -12 e no de baixo daria 41.

Depois ele aborda cálculos de erros, como a descida do gradiente, vamos supor que a rede previu 0.3 pra algo que a previsão correta era 1, a descida do gradiente pede a resposta real - a resposta prevista e esse é o erro, no caso ela errou bastante, aí ela arruma os pesos que causaram esse erro e faz o mesmo processo. Pra isso ela usa o delta, que é o cálculo pra ver quanto ela deve alterar o neurônio, pra calcular o delta ela pega o erro e multiplica pela derivada da função de ativação que é calculada na saída do neurônio. O resultado é o quanto os pesos daquele neurônio devem ser ajustados.

E também existe o backpropagation que aplica os pesos nos neurônios corrigidos, primeiro rodamos a rede normalmente fazendo o cálculo dela pras entradas, e depois o erro, e o backpropagation volta pra cada neurônio e vendo o quanto aquele neurônio errou fazendo o delta e ajusta os pesos de cada um.

E também é abordado alguns termos usados em redes neurais:

Bias: É o valor inicial pro neurônio, por exemplo em recomendação de música o bias é o valor da música em si, se for uma música famosa ela tem um bias alto pois a maioria ouve então a chance de você ouvir e alta também, já se for uma música pouco popular o bias é baixo, ele é um impulso pro neurônio decidir.

Mean Squared Error: Pega os erros e eleva ao quadrado pra rede ver o valor e arrumar o máximo possível pois pra ele o erro foi gritante e também remove os negativos, faz a média de todos e manda pra rede.

Root Mean Squared Error: É a raiz quadrada do MSE, ele volta pro valor original.

Stochastic Gradient Descent: Diferente da descida tradicional que pega todos os dados e depois calcula o erro e atualiza, esse aqui já olha pra um dado, calcula e atualiza, aí vai pro próximo calcula e atualiza.

Mini-Batch Gradient Descent: O meio termo entre os dois anteriores, a gente define um tamanho de tipo 20 dados, ela lê os 20 e calcula a média dos erros aí depois pega mais 20 e assim vai indo.

E com isso ele vai pra parte prática do card, onde implementa redes neurais.

Breast Cancer

Aqui ele estava fazendo o modelo prever se o câncer era maligno ou benigno que é uma classificação binária, pra isso ele usa os dados de um .csv baixado da internet.

Ele divide os dados em treino e teste usando sklearn com uma porcentagem. Depois cria a rede neural densa com uma camada oculta com 16 neurônios que usa a função reLU onde vai passar os dados de entrada e mandar pra camada de saída que usa a função sigmóide pra definir se é 0 ou 1, ou seja benigno ou maligno.

Usando o mesmo dataset ele usa a validação cruzada que é basicamente o K-FOLD que aprendi alguns cards anteriores. Ele divide em 10 “pastas” de dados, onde ele treina com 9 pastas e testa com 1, e assim vai indo. E mostra os resultados diferentes que variam um pouco entre valores altos e baixos de acurácia e como ele varia dependendo do treino e teste, pra isso ele pega a média de todos os resultados que no meu caso deu 0.79.

E explica também um método pra reduzir o overfitting, que é quando a rede treinou muito e decorou os dados de treino e não consegue prever pra dados novos, pra isso é usado o dropout que na hora do treino a rede ignora alguns neurônios pra que ela não dependa de neurônios específicos.

Iris Dataset

Usando o dataset Íris que é um dataset com dados da flor íris, aqui a rede precisa ver de qual espécie a iris é, como a rede não entende o nome da espécie ele define cada espécie como um conjunto de números usando tabelas na matriz.

Especie	A	B	C
Setosa	1	0	0
Versicolor	0	1	0
Virginica	0	0	1

Aqui está um exemplo visual, ele define assim porque se a gente define que setosa é 0 versicolor é 1 e virginica é 2 o computador entenderia que a virginica é mais importante que a setosa ou versicolor porem isso nao existe, ai ele coloca um código pra cada uma.

Ele separa os dados em treino e teste de novo com uma porcentagem, cria a rede neural com duas camadas ocultas com 4 neurônios cada usando a função reLU e a camada de saída usa a função softmax, que retorna uma % pra cada espécie.

Carros Usados Dataset

No dataset de carro ele queria prever o valor da venda de um carro, então é usada a regressão pra fazer isso, que também foi visto em cards passados. Ele começa limpando o dataset pois tinha muitas colunas desnecessárias e desorganizadas, ele remove colunas desnecessárias como data de criação código postal e etc..., e usa o mesmo esquema explicado anteriormente pra cada marca de carro pra nao gerar hierarquia e preenche valores que estão com NaN. Na parte da criação da rede ele coloca 2 camadas oculta usando reLU e a camada de saída usando a função de ativação linear.

Video Game Dataset

Aqui ele quer prever o valor de vendas nos EUA, Europa e Japão, como ele quer algo específico ele começa removendo colunas e linhas desnecessárias ou que estão duplicadas. Usa o mesmo método pra codificar a plataforma, gênero, a empresa que publicou o jogo e o rating do jogo. Criando a rede ele usa 3 camadas de saída pois ele quer prever pra cada região e usa a função relu para as camadas ocultas e a função linear para as de saída.

Prática

Usei um dataset chamado Pima Indians Diabetes Database que guarda dados sobre pacientes com diabetes ou não pra usar no machine learning pra prever se o paciente é diabetico ou não e foi exatamente o que tentei fazer na prática.

3. Conclusão

Em resumo, o card mostrou implementações de redes neurais que se mostram eficazes e úteis pra diversas coisas, ele dividiu bem a parte teórica e prática sendo bem equilibrado o professor tem uma didática boa e uma fala tão boa quanto. Tudo que foi abordado nesse card ajudou a fixar ainda mais o conhecimento introdutório que eu já tinha sobre redes neurais.